

Reviewer Pack — Minimal Rank of Geometric Langlands Duals vs Read-Twice BP D...

Entry 2da7cc936faf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 08:45:25 UTC

Minimal Rank of Geometric Langlands Duals vs Read-Twice BP Distinguishing Tensor Width

Entry ID: 2da7cc936faf **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 08:45:25 UTC

1. Conjecture

Field A (mathematical branch): Geometric Langlands Theory **Field B** (complexity object): Branching Program: read-once vs read-twice (BP)

Statement:

{‘st1’: ‘For every read-twice branching program P , let $R(P)$ denote the geometric Langlands dual of its underlying vector space.’, ‘st2’: ‘Then, the distinguishing tensor width $\rho(P)$ for P is lower-bounded by the minimal rank $m(R(P))$ with respect to the geometric Langlands correspondence, such that $\rho(P) = O(\log(|P|))$ but $\rho(IP_2 \text{ trivial BP}) = \Omega(n)$.’, ‘st3’: ‘Equivalently, for a constant c in $\mathbb{R}$, if P is a read-twice BP of size $|P|$ and IP_2 is the trivial BP, then $m(R(P))$ satisfies $m(R(P)) \geq c \cdot \log(|P|)$ but $m(R(IP_2)) \leq 1$.’}

Rationale (proposer’s reasoning):

{‘st1’: ‘Geometric Langlands theory provides a bridge between algebraic geometry and representation theory that may offer new insights into the structure of branching programs.’, ‘st2’: ‘Minimal ranks in geometric Langlands theory have been studied for their connections to number theory and arithmetic geometry, suggesting potential for complexity-theoretic applications.’, ‘st3’: ‘The conjecture aims to exploit this mathematical framework to provide a quantitative relationship between geometric Langlands and the distinguishing tensor width, potentially leading to new lower bounds in computational complexity.’}

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d9c8b7235db803ba

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated read-twice branching programs P of size $n \leq 40$, the ratio between distinguishing tensor width $\rho(P)$ and minimal rank $m(R(P))$

meets the following conditions: $\rho(P)/m(R(P)) \leq \log(|P|)/\log(c)$, where c is a constant. The conjecture is falsified if any seed produces a ratio greater than this threshold.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	SAFE	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Geometric Langlands duals" AND "read-twice branching program" - "minimal rank" AND "geometric Langlands correspondence" AND "tensor width" - "BP distinguishing tensor width" AND " $\log(|P|)$ " AND "geometric Langlands dual"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i + 1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C
```

```

def rank(A):
    A_rref = gaussian_elimination(A)
    rank = 0
    for row in A_rref:
        if any(row):
            rank += 1
    return rank

def read_twice_bp_size(bp):
    return len(bp) * (len(bp[0]) + 1)

def geometric_langlands_dual(bp):
    n = read_twice_bp_size(bp)
    V = [[i, j] for i in range(n) for j in range(n)]
    A = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    return rank(A)

def distinguishing_tensor_width(bp):
    n = read_twice_bp_size(bp)
    return math.log(n, 2)

c = 2
bp = [[random.randint(0, 1) for _ in range(random.randint(5, 10))] for _ in range(random.randint(5, 10))]
m_RP = geometric_langlands_dual(bp)
rho_P = distinguishing_tensor_width(bp)

return {
    "metric_name": "rho(P)/m(R(P))",
    "metric_value": rho_P / m_RP,
    "instances_tested": 1,
    "conjecture_holds": rho_P / m_RP <= math.log(read_twice_bp_size(bp), c) / math.log(c),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric = sum(r["metric_value"] for r in results) / len(results)
    std_metric = math.sqrt(sum((r["metric_value"] - mean_metric) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_369d898a.py", line 80, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_369d898a.py", line 64, in run_trial
    m_RP = geometric_langlands_dual(bp)
           ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_369d898a.py", line 56, in geometric_langlands_dual
    return rank(A)
           ^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_369d898a.py", line 42, in rank
    A_rref = gaussian_elimination(A)
             ^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_369d898a.py", line 27, in gaussian_elimination
    factor = A[j][i] / A[i][i]
             ^^^^^^^^
```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error, which prevented the generation of data necessary to evaluate the conjecture. | next: Investigate and fix the division by zero error in the code to allow for the completion of the test.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13161
2	preregistration	ollama_remote	glm4:latest	0	0	5990
3	novelty	ollama_remote	glm4:latest	0	0	5051
4	novelty	ollama_remote	glm4:latest	0	0	5592
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13698
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13958
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11514
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11815

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	15846

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 96623 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/2da7cc936faf.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2da7cc936faf.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2da7cc936faf.tar.gz (if generated)